EXPRESS CAR - Full Application Technical Report (Gujarati)

તારીખ: 27 April 2026
Project Name: Express Car
Platform: Flutter (Android/iOS/Web/Desktop capable)
Current Workspace Root: C:/EXPRESS_CAR

1) Project શું છે? (Overview)

Express Car એક multi-role car rental application છે, જેમાં મુખ્ય 3 user persona cover થાય છે:

1. સામાન્ય User (car book કરનાર)
2. Owner/Business (cars/fleet manage કરનાર)
3. Admin (system-level monitoring અને control)

આ app માં complete flow છે:

- Authentication
- Role-based routing
- Car listing + search + favorites
- Booking + booking details + history
- Payment state management (pending / paid / failed)
- Razorpay online payment integration
- Live location tracking
- Profile management
- Admin + owner operation modules

2) High-Level Architecture

Frontend Layer

- Flutter UI screens (`lib/`)
- State handling mostly StatefulWidget/StreamBuilder based
- Reusable services/widgets under `lib/services/`

Backend/Data Layer

- Firebase Authentication (user auth)
- Cloud Firestore (primary database)
- Firebase Storage (media uploads)
- Firebase Cloud Functions (`functions/index.js`) for secure payment operations

Payment Layer

- Razorpay Checkout SDK (`razorpay_flutter` in app)
- Server-side order creation + signature verification in Cloud Functions

3) Core Technologies Used

1. Flutter + Dart
2. Firebase Core
3. Firebase Auth
4. Cloud Firestore
5. Firebase Storage
6. Google Sign-In
7. Razorpay (mobile SDK + node backend SDK)
8. HTTP package (API calls)
9. Geolocator (device location)
10. flutter_map + latlong2 (map rendering)
11. Cloudinary + ImgBB (image/document pipelines in specific modules)

4) Dependencies (pubspec.yaml મુજબ)

Runtime dependencies

1. flutter
2. firebase_core
3. firebase_auth
4. cloud_firestore
5. firebase_storage
6. google_sign_in
7. pinput
8. http
9. image_picker
10. multi_select_flutter
11. flutter_launcher_icons
12. cupertino_icons
13. cloudinary_public
14. geolocator
15. flutter_map
16. latlong2
17. razorpay_flutter

Dev dependencies

1. flutter_test
2. flutter_lints

5) Folder Structure Summary

1. `lib/` - complete Flutter app source
2. `functions/` - Firebase Cloud Functions (Node.js backend)
3. `assets/images/` - bundled image assets
4. `docs/` - documentation + HTML + PDF report generator script

5. `android/`, `ios/`, `web/`, `windows/`, `linux/`, `macos/` - platform projects

6) Authentication & Routing Flow

Main routing logic `AuthWrapper` based છે:

1. App start -> splash
2. Firebase auth state check
3. Logged-in user માટે Firestore users doc પરથી role/profile check
4. Route decide:
   - admin -> admin panel
   - incomplete profile -> complete profile page
   - normal user -> home page

Authentication capabilities:

1. Email/password signup/login
2. Google sign-in
3. Email verification flow
4. Password reset flow

7) Main Feature Modules

A) User Side

1. Car browsing & category filtering
2. Favorites management
3. Booking creation with date range and rental units
4. Booking details page
5. Booked cars/history page
6. Live location tracking page
7. Rate/review (only paid booking પછી)

B) Owner/Business Side

1. Add car
2. Manage cars
3. Manage bookings
4. Owner dashboard

C) Admin Side

1. Admin panel dashboard
2. Manage users
3. Fleet/driver document handling

8) Database (Cloud Firestore) - Collections & Purpose

Project rules અને code મુજબ મુખ્ય collections:

1. `users`
   - role (`user`/`admin`)
   - profile fields
   - auth-linked identity data

2. `cars`
   - car inventory
   - owner linkage (`owner_uid`)
   - pricing/details/images

3. `fleet_items`
   - approved/manageable fleet entries (admin controlled)

4. `bookings`
   - booking identity + dates + car info + userId
   - payment fields (`payment_status`, `payment_id`, `payment_order_id`, etc.)
   - review fields (`user_rating`, `user_review`, `reviewed_at`)

5. `driver_documents`
   - driver/fleet supporting docs (admin scope)

6. `counters`
   - incremental counters જેમ કે `booking_counter`, `car_counter`

9) Firestore Security Rules - Important Logic

`firestore.rules` માં key protections:

1. `isSignedIn`, `isAdmin`, `isSelf` helper functions
2. Bookings create only signed-in owner of that booking માટે
3. Booking review update strictly constrained (allowed fields only)
4. Cars create/update/delete owner અથવા admin-only
5. Fleet + driver_documents admin-only write access
6. Role escalation prevent કરવા users rulesમાં checks

10) Payment System - Razorpay Integration (Current)

App-side (`lib/HomeDetails/Booking/payment_page.dart`)

PaymentPage માં નીચેની online payment pipeline છે:

1. `PAYMENT_CREATE_ORDER_URL` પર authenticated POST -> order create
2. Razorpay SDK checkout open
3. success callback -> `PAYMENT_VERIFY_URL` call
4. failure callback -> `PAYMENT_FAILURE_URL` call
5. તમામ backend callsમાં Firebase ID token `Authorization: Bearer <token>` સાથે મોકલાય છે

`dart-define` keys used:

1. `RAZORPAY_KEY_ID`
2. `PAYMENT_CREATE_ORDER_URL`
3. `PAYMENT_VERIFY_URL`
4. `PAYMENT_FAILURE_URL`

Backend-side (`functions/index.js`)

Cloud Functions endpoints:

1. `createRazorpayOrder`
   - booking ownership verify
   - Razorpay order create
   - booking payment fields pending stateમાં set

2. `verifyRazorpayPayment`
   - HMAC signature verify
   - booking `payment_status: paid` update
   - payment refs/time store

3. `markPaymentFailed`
   - booking `payment_status: failed` update
   - failure reason store

Booking Payment States

1. `pending` - booking create પછી initial state
2. `paid` - verification successful પછી
3. `failed` - payment fail handler પછી

11) Rating/Review Control Logic

Booking review હવે payment-gated છે:

1. UI side પર unpaid booking માટે rating button disabled/informational
2. Save સમયે live booking doc ફરીથી read થાય છે
3. `payment_status != paid` હોય તો review save reject થાય છે

આ validation `booking_details_page.dart` માં runtime checkથી enforced છે.

12) APIs/Services - શું માટે વપરાય છે?

1. Firebase Auth API
   Use: login/signup, auth state, email verification, password reset

2. Firestore API

   Use: users/cars/bookings/fleet/documents/counters CRUD + stream-based UI

3. Firebase Storage API
   Use: profile image વગેરે uploads

4. Google Sign-In API
   Use: social auth

5. Razorpay API/SDK
   Use: secure online payments, order creation, signature-based verification

6. Geolocator API
   Use: location permissions + current/live location

7. flutter_map + latlong2
   Use: live tracking map UI rendering

8. Cloudinary / ImgBB API
   Use: selected image/document upload flows

13) Important Backend Config Files

1. `firebase.json` - firebase project service config
2. `firestore.rules` - DB access rules
3. `storage.rules` - storage access rules
4. `functions/package.json` - cloud functions dependencies/scripts
5. `functions/.env` - Razorpay secrets (local/runtime env)

14) Cloud Functions Runtime Info

`functions/package.json` મુજબ:

1. Node engine target: `20`
2. dependencies:
   - `firebase-admin`
   - `firebase-functions`
   - `razorpay`

15) Deployment/Run Commands (Reference)

Flutter

1. `flutter pub get`
2. `flutter run --dart-define=RAZORPAY_KEY_ID=... --dart-define=PAYMENT_CREATE_ORDER_URL=...
--dart-define=PAYMENT_VERIFY_URL=... --dart-define=PAYMENT_FAILURE_URL=...`

Functions

1. `cd functions`
2. `npm install`
3. `.env` set કરો:
   - `RAZORPAY_KEY_ID=...`
   - `RAZORPAY_KEY_SECRET=...`
4. deploy:
   - `firebase deploy --only functions --project <project-id>`

નોંધ: Cloud Functions v2 deploy માટે Firebase Blaze plan જરૂરી પડી શકે.

16) What Code is Used for What? (Quick Mapping)

1. `lib/main.dart` - app startup + Firebase init
2. `lib/Authentication/*` - auth pages + routing
3. `lib/HomeDetails/Home_Page/*` - home, car models/data, listing
4. `lib/HomeDetails/Booking/Book_car.dart` - booking creation
5. `lib/HomeDetails/Booking/payment_page.dart` - Razorpay + payment actions
6. `lib/HomeDetails/Booking/booking_details_page.dart` - booking summary + rating save checks
7. `lib/HomeDetails/Booking/Booked_Car.dart` - booked car list/payment status display
8. `lib/HomeDetails/Menu/*` - profile/menu/account actions
9. `lib/Handle_Car/*` - owner/business management screens
10. `lib/Admin/*` - admin management screens
11. `lib/services/*` - helper services (presence, tracking, upload, image widget)
12. `functions/index.js` - payment backend endpoints

17) Security Summary

1. Payment verification server-side HMACથી થાય છે (client trust નથી)
2. Booking-payment updates backend ownership check સાથે છે
3. Firestore rules દ્વારા unauthorized update restrictions છે
4. Review save માટે paid status runtime check enforced છે

18) Project Strengths (Viva Points)

1. End-to-end role-based architecture
2. Firebase-native secure auth/data model
3. Payment flowમાં client + backend બંને level validation
4. Real-time friendly design (streams/location)
5. Modular code organization (`Authentication`, `HomeDetails`, `Handle_Car`, `Admin`, `services`)

19) Known Operational Notes

1. Functions deploy સમયે project billing plan dependency આવી શકે
2. Razorpay keys/env proper set ન હોય તો order create fail થશે
3. `dart-define` URLs missing હોય તો app payment page error બતાવે છે

20) Final Conclusion

Express Car app production-style architecture સાથે build કરાયેલ Flutter + Firebase solution છે. આમાં authentication, role-based authorization, booking lifecycle, payment security, and review integrity જેવા critical use-cases cover થાય છે.
Viva માટે આ project end-to-end full-stack mobile app example તરીકે strong છે.